

EU AI Act Compliance Navigator — 项目开发全纪实

本文档记录项目从零到 v1.0 的完整开发历程，包括技术选型、遇到的问题、解决过程和关键决策。用途：为 UCL LLM (Technology Law) 申请文书提供真实素材。

一、项目起点：从一个问题开始

1.1 我想解决什么

EU AI Act（欧盟人工智能法案，Regulation 2024/1689）是全球首部综合性 AI 监管法律，将于 2026 年 8 月 2 日全面执行。这部法律有 180 条序言、113 条正文、13 个附件，企业面临三个现实困难：我的 AI 系统算不算高风险？需要满足哪些合规义务？这些义务散落在哪些条文里？

我决定构建一个工具：用户用自然语言描述自己的 AI 系统，工具自动检索相关法律条文，生成结构化的合规评估报告。

1.2 我的起点

这不是我第一次接触 RAG 技术。此前我已经用 Ollama + DeepSeek 14B + AnythingLLM 搭建过一个法律文档 RAG 系统，对向量检索、文本切分、Prompt 工程有基本理解。但那个系统是通用的“法律问答”，没有针对特定法规的结构化处理。这一次，我要解决一个具体的监管问题。

1.3 从 0 到 v0.1：先做一个能跑通的最小闭环

从 0 到 v0.1，我先把问题缩小到一个最小可验证命题：如果用户输入一个明确的法条级问题，系统能否在浏览器里返回对应条文片段，并且不丢失法律层级。第一个版本并不追求直接输出完整的“合规评估报告”，而是先要求形成一条最小闭环：原始 PDF → 文本解析 → 法律结构切分 → 本地检索 → Streamlit 可视化。

因此，最早的工作不是引入模型，而是搭建项目骨架：用 `data/raw` 与 `data/parsed` 分离原始文件和解析结果；用 `src/ingest.py` 负责把 PDF 转成可处理文本；用 `src/legal_chunks.py` 识别 Chapter、Section、Article、Annex、Recital 的边界；让 `app.py` 只承担最小展示与查询入口。只有这个最小闭环先跑通，后续任何 Embedding、向量数据库或 LLM 升级才有意义。

这一阶段我也开始把 Codex 用作终端内工程辅助工具：它帮助我在项目目录内读取文件、生成局部修改、执行命令并快速验证最小修改是否破坏原型。我的角色不是“接受输出”，而是先定义目标与边界，再逐步让代码达到我要求的可解释状态。

二、第一阶段：框架搭建 (v0.1 - v0.2)

2.1 技术选择：为什么从最简单的方案开始

我没有一上来就用 LangChain 或 LlamaIndex 这些重型框架。我的第一个决策是：**用最简单的技术验证核心想法是否可行。**

最初的技术栈：

- **文档解析**：pypdf（基础 PDF 文本提取）
- **文本切分**：自定义的法律结构感知切分器（不是固定长度切分）
- **检索**：纯关键词匹配 + 规则加权（没有向量数据库，没有 Embedding）
- **前端**：Streamlit
- **LLM**：无（纯规则拼接答案）

这个选择是刻意的。法律文本有很强的结构性——Chapter、Section、Article、Annex、Recital 都有明确的边界标记。如果连关键词匹配都做不好，上向量检索也没用。

我把 v0.1-v0.2 视为一个刻意控制变量的验证阶段，而不是功能尚少的过渡版本。目标不是尽快叠加 Embedding、向量数据库和 LLM，而是先确认 EU AI Act 这类强结构法律文本能否被稳定切分、定位和引用。只有在这个基线成立之后，后续引入语义检索与生成层，才是可解释的升级，而不是用更复杂的技术掩盖前期问题。

2.2 法律结构感知切分：第一个技术挑战

通用 RAG 系统通常按固定长度（如 512 tokens）切分文本。但法律文本不能这么做。

问题：EU AI Act 的 Article 6（高风险分类规则）包含条件和例外，如果从中间截断，检索到的片段会丢失关键的“除外条款”，导致合规判断错误。

我的解决方案：在 `src/legal_chunks.py` 中实现了法律结构感知的切分器。它识别 Chapter、Section、Article、Annex、Recital 的边界标记，按法律逻辑单元切分，而不是按字符数。每个 chunk 自动标注元数据：条文编号（`article_number`）、附件引用（`annex_ref`）、序言编号（`recital_ref`）、所属章节（`chapter_heading`）。

这个切分器最终生成了 730 个结构化 chunks（后来接入 14 份法规文件后增长到 3230 个），每个 chunk 都携带完整的法律定位信息。

2.3 v0.1 - v0.2 的成果

到这个阶段，系统能够：

- 解析 EU AI Act 全文 PDF

- 按法律结构切分为带元数据的 chunks
- 用关键词匹配检索相关条文
- 在 Streamlit 上展示检索结果

为了避免后续把性能变化误归因于新模型或新框架，我在这一阶段反复检查 Article、Annex、Recital 这类显式引用问题是否能稳定返回正确片段，并逐步补上 metadata 展示和最小回归验证。这样，后续 v0.3 引入语义检索时，性能变化才可以被更清楚地归因。

前期阶段的价值不在于“功能多少”，而在于建立一个可检验的法律结构基线：Article、Annex、Recital 能否被稳定识别，除外条款能否不被截断，宽泛问题能否至少被路由到正确的法律单元。后续所有升级都以这个基线为参照。

局限：没有语义理解能力。用户问“我的 HR 筛选工具是否合规”，系统无法理解这和“recruitment”、“Annex III”的关联——因为用户的描述里没有出现这些关键词。

三、第二阶段：引入向量检索 (v0.3)

3.1 关键技术决策：OpenRouter + OpenAI SDK

要做语义检索，需要 Embedding 模型将文本转为向量。我选择了 OpenRouter 作为 API 网关，通过它调用 OpenAI 的 text-embedding-3-small 模型。

遇到的第一个大坑：OpenRouter 有一个官方 Python 包叫 `openrouter`，我最初用了它。结果：

- `from openrouter import Client` → `ImportError`
- `or_client.embeddings(text)` → `TypeError`
- `or_client.embeddings.create(...)` → `AttributeError`
- `or_client.embed_text(...)` → `AttributeError`

花了整整一个上午尝试各种版本和写法，全部失败。

解决方案：放弃 `openrouter` 包，改用 `openai` 包，只把 `base_url` 指向 OpenRouter：

```
from openai import OpenAI
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
resp = client.embeddings.create(model="openai/text-embedding-3-small", input=text
```

这三行代码解决了一上午的问题。**教训：**第三方封装不一定比原生 SDK 好用。OpenRouter 的 API 完全兼容 OpenAI 格式，直接用 OpenAI SDK 是最稳定的方式。

3.2 另一个隐蔽的 bug: ingest() 的返回值

我在 Python REPL 里手动跑管道时，用 `docs = ingest()` 获取文档列表，然后尝试 `docs_texts = [doc["text"] for doc in docs]` 提取文本。结果报 `KeyError: 'text'`。

原因：`ingest()` 返回的不是文本内容，而是文件处理日志。它的 `output` 字段是输出文件名（如“EU AI Act...txt”），不是文本本身。真正的文本被写到了 `data/parsed/` 目录。我一直在用文件名当文本去做 `build_structured_chunks`，所以只产生 1 个 chunk。

解决方案：写了一个完整的管道脚本 `run_pipeline_chroma.py`，从 `data/parsed/` 目录读取文件内容，而不是依赖 `ingest()` 的返回值。

教训：不要在 Python REPL 里粘贴多行代码（尤其是含中文注释的），写成 `.py` 文件执行更可靠。

3.3 向量化 + ChromaDB 存储

管道跑通后，730 个 chunks 全部向量化并存入 ChromaDB。测试查询“What are the obligations for providers of high-risk AI systems?” 返回了 Article 16（高风险 AI 系统提供者义务），排在第一位。

3.4 接入 LLM 生成合规评估

检索管道跑通后，接入 GPT-4o-mini 生成结构化的合规评估报告。Prompt 设计包含四项强制约束：

1. 每条声明必须引用具体条文号
2. 不得省略除外条款
3. 边界案例必须保守分类
4. 不得使用检索上下文之外的知识

v0.3 的成果：完整的 RAG 闭环——语义检索 + LLM 合规评估 + Streamlit 前端。

四、第三阶段：检索精度优化 (v0.4)

4.1 发现问题：Annex III 和 Annex IV 的混淆

纯向量检索有一个致命问题：Embedding 模型通过语义相似度检索，但“Annex III”（高风险 AI

系统清单) 和 “Annex IV” (技术文档要求) 在向量空间中几乎重叠——它们的文本结构相似, 只是编号不同。但法律后果完全互斥。

如果用户问“我的系统是否属于 Annex III 高风险?”, 系统可能混淆召回 Annex IV 的内容, 导致合规判断错误。

4.2 解决方案: Self-Query 路由

我在查询引擎前加了一个路由层: 用正则表达式检测用户查询中是否包含明确的法条引用 (Article X、Annex X、Recital X)。如果检测到, 就自动构建元数据过滤条件, 只在匹配的 chunks 中做向量检索; 如果没有检测到, 就走常规全量向量检索。

最初我尝试用 LLM 做法条提取 (让 GPT-4o-mini 判断用户查询中是否包含法条引用)。但在 Streamlit Cloud 上部署后发现 OpenRouter 不稳定地支持 `response_format=json_object` 参数, 导致 LLM 调用静默失败。

最终方案: 改用纯正则表达式, 零延迟、100% 可靠。对于 “Article 9”、“Annex III” 这类明确引用, 正则的检测准确率是 100%。

4.3 动态 Token 预算

法律结构切分后, 不同 chunk 的长度方差很大。一个 Annex 表格可能数百 tokens, 一条禁止规定可能几十 tokens。固定检索 8 个 chunks 不合理——8 个长 chunk 可能超出 LLM 上下文窗口, 8 个短 chunk 可能不足以覆盖完整的合规分析。

解决方案: 自动调整 Token 预算。默认 6000 tokens, 如果查询涉及多个法条/附件 (如 “Article 6 and Annex III”), 自动提升到 10000; 如果是宽泛的义务类查询, 提升到 8000。在预算内按相关性排序填充 chunks。

五、第四阶段: 产品化与部署 (v0.4 → v1.0)

5.1 聊天界面

将传统的表单提交界面 (输入框 + 按钮 + 结果展示) 改为对话式聊天界面。用户的问题和 AI 的回答像聊天记录一样排列, 支持多轮对话。

Streamlit 的 `st.chat_message` 组件默认所有消息左对齐。为了让用户消息靠右 (像微信/iMessage), 我用自定义 HTML + CSS 实现了蓝色右对齐气泡。

5.2 多语言支持

添加了 6 种语言支持: 英语、法语、德语、西班牙语、简体中文、繁体中文。实现方式是在 LLM 的 system prompt 开头注入语言指令 (如 “请用简体中文回答”), 同时 UI 元素 (标题、按钮文

字、disclaimer) 从语言字典中读取。

一个细节问题：如果用户先用英语提问得到回答，然后切换到中文，已生成的英语回答不会自动变成中文。解决方案是在每条回答旁加了“用当前语言重新生成”按钮。

5.3 云端部署：ChromaDB 兼容性问题

本地开发用 ChromaDB 作为向量数据库，一切正常。但部署到 Streamlit Community Cloud 后遇到严重兼容性问题：

- Streamlit Cloud 使用 Python 3.14 (太新)
- ChromaDB 依赖的 protobuf 和 pydantic 在 3.14 上报 TypeError / ConfigError
- 尝试锁定版本、添加 .python-version 文件，均未解决

最终方案：彻底放弃 ChromaDB 在云端的使用。将所有向量数据导出为 JSON 文件 (vector_store.json)，在云端用纯 Python 实现余弦相似度检索。这样零外部依赖，完全兼容任何 Python 版本。

导出时遇到两个小问题：

1. ChromaDB 返回的 embeddings 是 numpy 数组，json.dump 不支持 → 用 .tolist() 转换
2. 导出的 JSON 文件 108MB，超过 GitHub 的 100MB 限制 → 用 round(x, 5) 截短 embedding 精度到 5 位小数，文件降至 43MB

5.4 虚拟环境反复崩溃

开发过程中虚拟环境崩溃了三次：

1. 第一次：ChatGPT 建议用 `rm -rf venv` 然后 `python3 -m venv venv` 重建，但系统默认 python3 指向 3.14，新 venv 装的包全不兼容
2. 第二次：反复安装/卸载 streamlit 和 openai 导致 anyio 包损坏（版本号变成“4.13.0 2”），pip 自己都无法运行
3. 第三次：装了 openrouter 包后卸载，残留依赖污染了环境

每次的解决方案都一样：

```
rm -rf venv
python3.12 -m venv venv # 必须指定 3.12
source venv/bin/activate
pip install streamlit openai chromadb pypdf
```

教训：创建虚拟环境时必须显式指定 Python 版本，不能依赖系统默认的 `python3`。

5.5 API Key 管理

开发中 API key 出过几次问题：

- 两条终端命令粘在一行执行（`export KEY=... python script.py`），导致 `export` 失败
- 新开终端标签页后忘记重新设置环境变量
- Streamlit Cloud 上忘记在 Secrets 里配置 key

最终在 `app_chroma.py` 中实现了三层读取：

```
api_key = os.environ.get("OPENROUTER_API_KEY", "") # 本地环境变量
if not api_key:
    api_key = st.secrets.get("OPENROUTER_API_KEY", "") # 云端 Secrets
if not api_key:
    st.error("Set OPENROUTER_API_KEY.") # 明确报错
```

六、架构审计：多模型交叉验证

6.1 过程

项目开发过程中，我让 ChatGPT 和 Claude 分别审查架构设计，然后交叉验证各自提出的问题。

第一轮审计（ChatGPT → Claude 实现）：

- 漏洞 1：pypdf + 固定长度切分破坏法律层级 → 引入 LlamaParse + MarkdownNodeParser（设计文档层面）
- 漏洞 2：全局元数据注入导致节点污染 → 改为节点级正则提取
- 漏洞 3：tree_summarize 模式导致法律幻觉 → 改为 compact 模式

第二轮审计（ChatGPT）：

- H3 子节点元数据失联 → 父级标题继承机制
- 固定 top_k 与节点长度方差冲突 → 动态 Token 预算
- 向量检索对法条编号的语义盲区 → Self-Query 路由

第三轮审计（ChatGPT）：

- ChromaDB 不支持字符串 \$contains → 改用列表存储 + \$eq 精确匹配

- Self-Query 的 JSON 输出不稳定 → 改用 OpenAI 原生 JSON 模式
- FilterCondition.OR 在旧版 ChromaDB 不支持 → 单条件不使用 OR

6.2 这个过程教会我什么

多模型交叉审计是一种有效的架构验证方法。每个模型有不同的关注点：ChatGPT 倾向于给出完整方案但有时过度设计，也擅长把问题拆成可验证的小步并发现原型里的逻辑缺口；Claude 适合做最终实现和代码生产；Codex 则像终端内的执行助手，负责在项目目录内读取文件、生成局部修改、执行命令和回归检查，把”修改—运行—检查”这一循环压缩成可控的小步迭代。

但最重要的是：**不能无条件信任任何一个模型的建议**。ChatGPT 让我用已废弃的 openai v0.x 语法，让我安装不稳定的 openrouter 包，让我用 chromadb.api.Client（不存在的类）做测试——每一次都导致了新的报错。每个建议都需要验证。

七、最终技术栈

层级	技术	选择理由
文档解析	pypdf + 自定义法律切分	结构感知切分优于固定长度；不依赖云服务
Embedding	text-embedding-3-small via OpenRouter	1536 维，精度足够；通过 OpenAI SDK 调用最稳定
向量存储	JSON 文件（云端）/ ChromaDB（本地）	JSON 零依赖、完全兼容；ChromaDB 仅本地开发用
检索策略	Self-Query 正则路由 + 余弦相似度 + 动态 Token 预算	正则零延迟 100% 可靠；Token 预算自动适配
LLM	GPT-4o-mini via OpenRouter	低成本、低延迟、128K 上下文窗口
Prompt	四项强制约束（引用/例外/保守/限定上下文）	法律场景防幻觉
前端	Streamlit（聊天界面）	Python 原生、部署简单
部署	Streamlit Community Cloud	免费、从 GitHub 自动部署
多语言	6 种（EN/FR/DE/ES/ZH-CN/ZH-TW）	System prompt 注入 + UI 字典

八、关键数字

指标	数值
法律文档	14 份（EU AI Act + 交叉法规）
结构化 chunks	3230 条
Embedding 维度	1536
开发周期	3 天（从框架到 v1.0 部署上线）
Git commits	24 次
版本标签	6 个（v0.1 → v1.0）
遇到的重大 bug	8 个（全部解决）

九、这个项目证明了什么

9.1 法律理解

我不是在做一个通用的 RAG demo。切分器识别 Chapter/Section/Article/Annex/Recital 的边界，Self-Query 路由区分 Annex III 和 Annex IV 的法律后果差异，Prompt 强制保留除外条款——这些设计都源于对 EU AI Act 法律结构的理解。

9.2 技术能力

从文档解析到向量化、从检索到生成、从本地开发到云端部署，整个 RAG 管道是我独立搭建的。遇到 OpenRouter SDK 不兼容、ChromaDB 版本冲突、虚拟环境崩溃、文件超限等问题时，我没有换方向，而是逐个诊断和解决。

前期 v0.1-v0.2 的结构化检索、元数据收尾和回归评估，也在与 ChatGPT / Codex 的持续协作中完成；关键不在于让模型“代写”，而在于把复杂问题拆成可验证的小改动，再逐步回归。

9.3 工程判断

我知道什么时候用简单方案（正则替代 LLM 做法条提取）、什么时候放弃不可靠的依赖（openrouter 包 → openai 包、ChromaDB 云端 → JSON 文件）、什么时候停止添加功能（ChatGPT 和 Claude 都不断建议新功能，但项目的核心价值不在视觉包装而在法律分析的准确性）。

9.4 这不是终点

设计文档中还有更多技术方向可以探索：LlamaParse 保留法律层级、MarkdownNodeParser 语义切分、双层元数据继承、BM25 混合检索。但对一个申请展示项目来说，当前版本已经完整地证明了“法律如何以可执行的方式嵌入技术系统”这一核心命题。

十、文书中可以怎么讲这个故事

10.1 一句话版本

“I built a RAG-based compliance navigator for the EU AI Act that retrieves relevant legal provisions from 3,230 indexed passages and generates structured risk assessments — not as a technical exercise, but as my answer to how law can be embedded into technology systems in an executable way.”

10.2 展开版本

“Building on my prior experience with legal RAG systems, I developed an EU AI Act Compliance Navigator — a tool where users describe their AI system in natural language and receive a structured compliance assessment grounded in specific articles, recitals, and annexes.

The technical challenge was not simply indexing legal text, but preserving its structure. Legal provisions have hierarchical dependencies — classification rules in Article 6 reference Annex III’s high-risk list, which in turn requires compliance obligations spread across Articles 9 through 14. A naive text-splitting approach would sever these connections. I designed a legal-structure-aware chunking system that respects Chapter, Section, Article, and Annex boundaries, and implemented a self-query routing mechanism that detects explicit legal references in user queries to enable precision retrieval.

The project went through three rounds of architecture review using multiple AI models as auditors, each round revealing deeper issues — from metadata contamination to embedding-level confusion between Annex III and Annex IV. The final system processes over 3,200 legal text passages in six languages, deployed as a publicly accessible web application.

This project is not a demonstration of technical sophistication for its own sake. It is my attempt to answer a question central to technology law: how can complex regulatory frameworks be made navigable through computational tools, without overclaiming what such tools can deliver?”
